

Technologie Sieciowe

Warstwa aplikacji

dr inż. Krzysztof Kosmowski

Mapa wykładu

- Zasady budowy protokołów w. aplikacji
- WWW i HTTP
- DNS
- Poczta elektroniczna
- SMTP, POP3, IMAP
- FTP
- Dystrybucja zawartości
- Schowki Internetowe
- Sieci dystrybucji zawartości

Aplikacje sieciowe: trochę słownictwa

Proces: program działający na hoście.

- na jednym hoście, dwa procesy komunikują się przez **komunikację międzyprocesową** (zdefiniowaną przez System Operacyjny).
- procesy działające na różnych hostach porozumiewają się **protokołem warstwy aplikacji**

agent (user agent):

- komunikuje się z użytkownikiem i siecią.
- implementuje interfejs użytkownika i protokół warstwy aplikacji
 - WWW: przeglądarka
 - E-mail: program pocztowy
 - streaming audio/video: odtwarzacz multimedialny

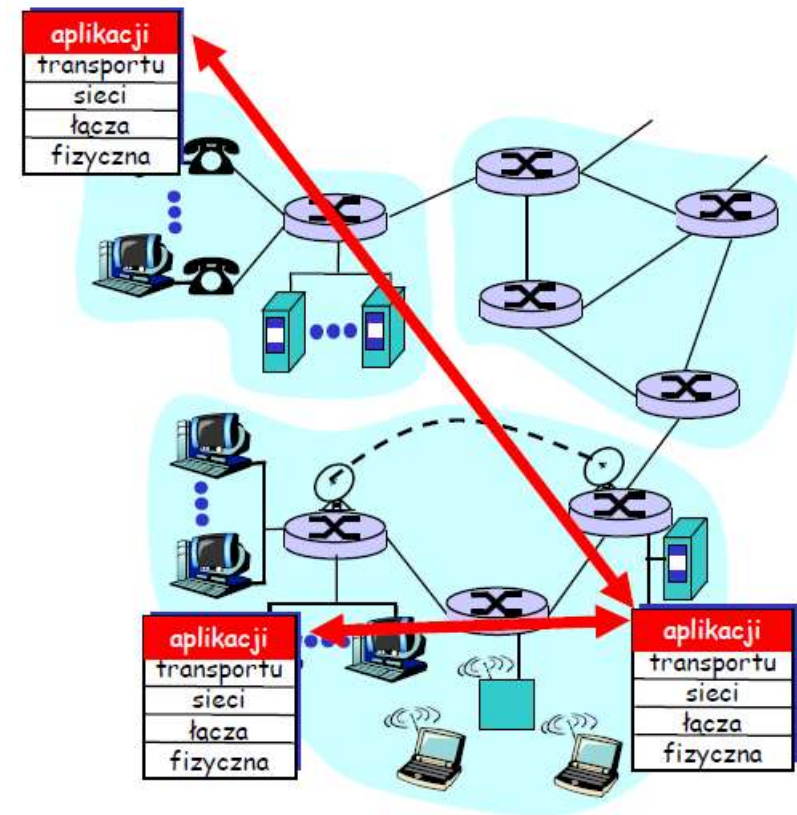
Aplikacje i protokoły warstwy aplikacji

Aplikacje: komunikujące, rozproszone procesy

- np., e-mail, WWW, dzielenie plików P2P, komunikatory
- działają na systemach końcowych (hostach)
- wymieniają komunikaty

Protokoły warstwy aplikacji

- "kawałek" aplikacji
- definiują komunikaty i akcje aplikacji
- używają usług komunikacyjnych niższej warstwy (TCP, UDP)



Protokół w. aplikacji określa...

- Rodzaje komunikatów, np., komunikaty żądania i odpowiedzi
- Składnię komunikatów: jakie pola w komunikatach i jak są oddzielane
- Znaczenie informacji w polach komunikatu
- Zasady określające kiedy i jak procesy wymieniają komunikaty

Protokoły publiczne:

- definiowane w dokumentach *Request For Comments* (RFC)
- pozwala na współpracę różnych systemów
- n.p., HTTP, SMTP

Protokoły prywatne:

- n.p., KaZaA

Model klient/serwer

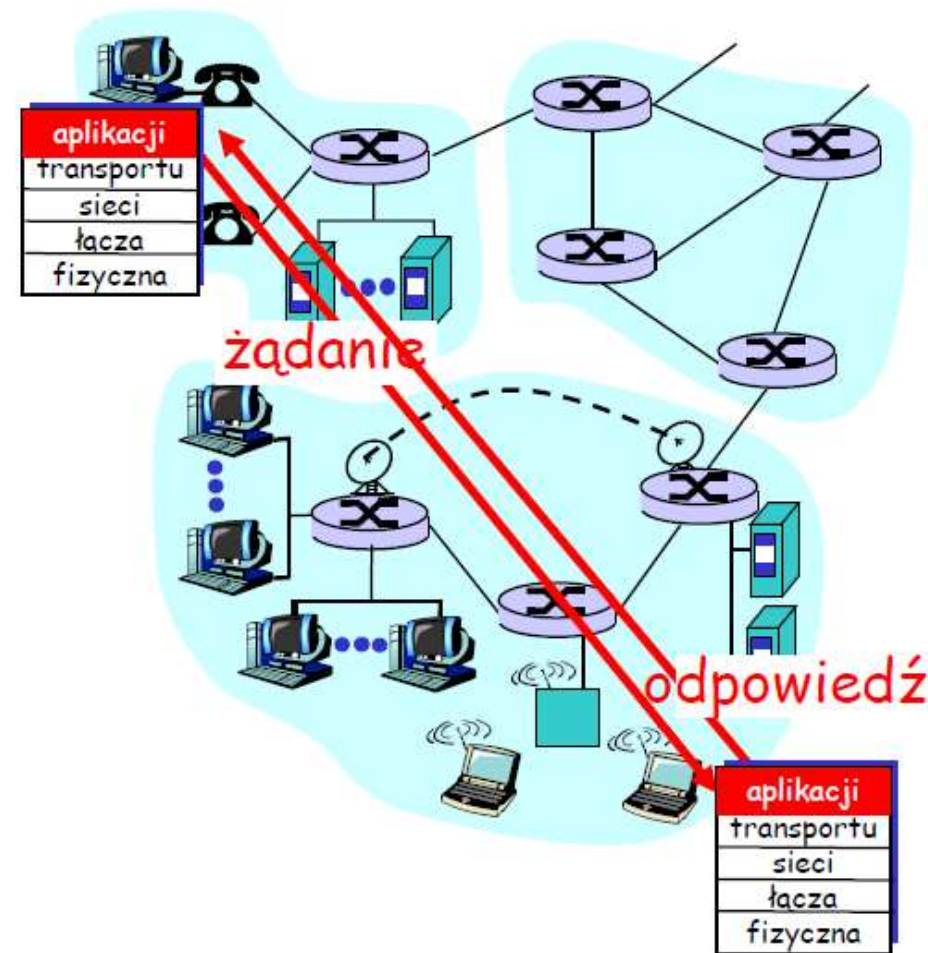
Typowa aplikacja sieciowa ma dwie części: *klienta* i *serwera*

Klient: żądanie

- rozpoczyna komunikację z serwerem (“mówi pierwszy”)
- zwykle prosi o usługę serwera,
- WWW: klient implementowany przez przeglądarkę; e-mail: przez program pocztowy

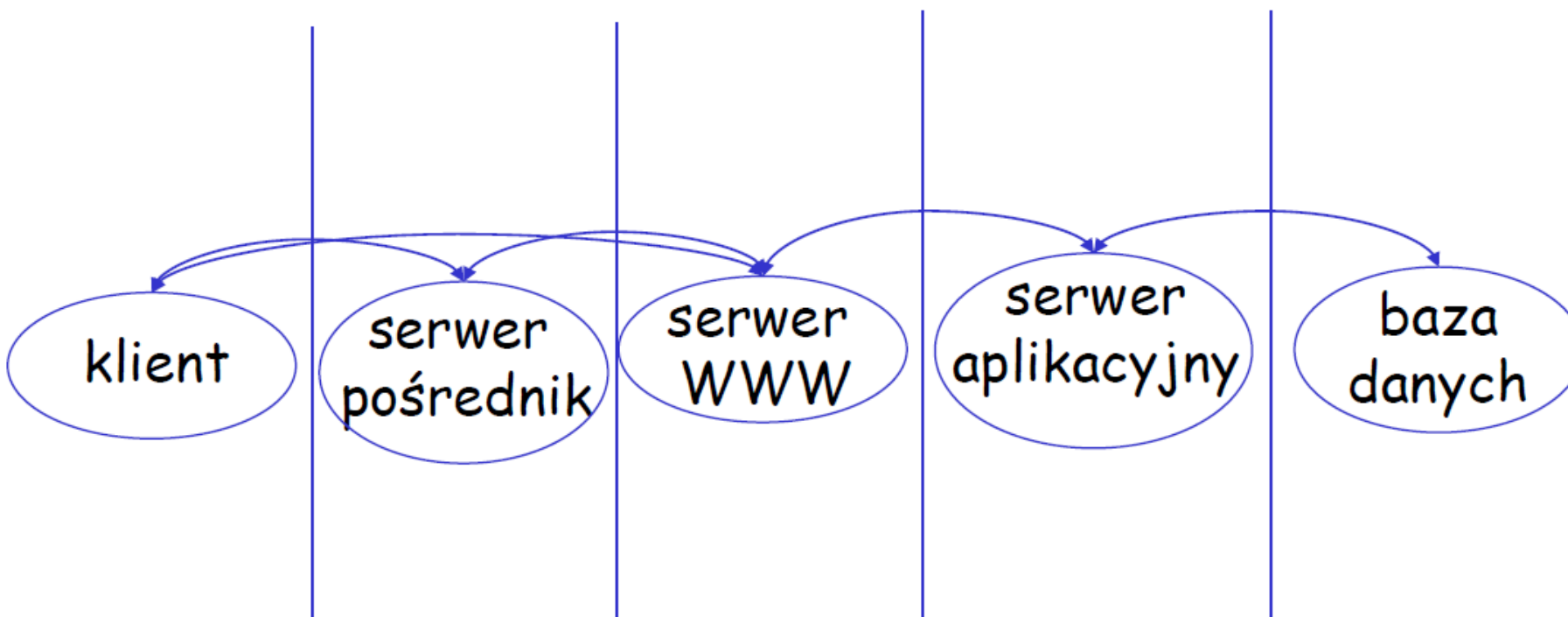
Serwer: odpowiedź

- udostępnia żadaną usługę klientowi
- n.p., serwer WWW wysyła żadaną stronę, serwer poczty dostarcza pocztę



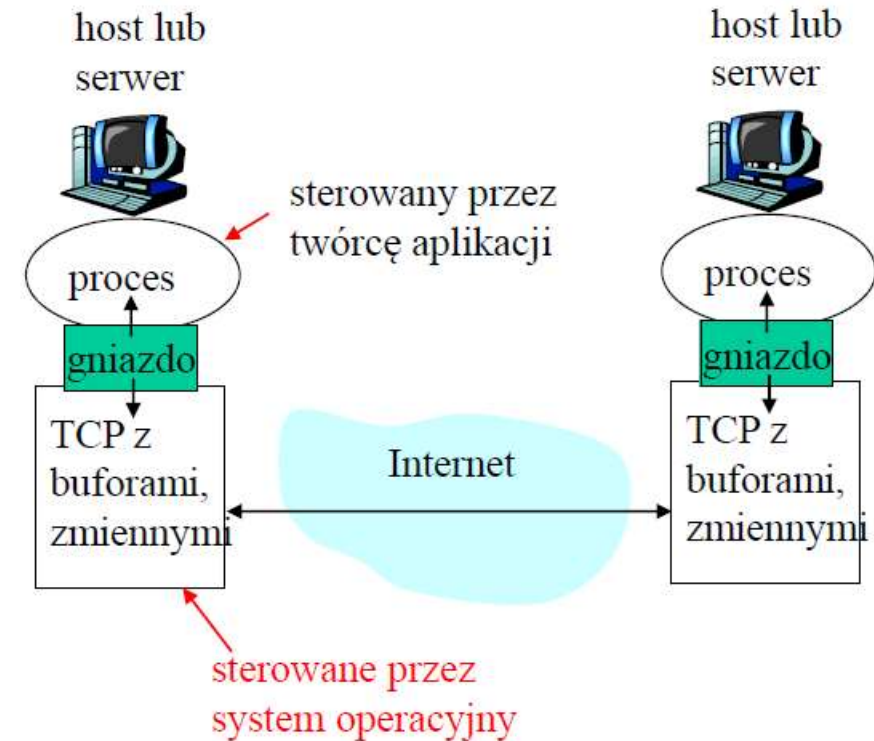
Rozwinięcia modelu klient/serwer

- Dodatkowe etapy pośrednie: płaszczyzny (ang. *multi-tier architecture*)



Procesy komunikujące przez sieć

- proces wysyła/odbiera komunikaty do/z swojego *gniazda*
- gniazdo można porównać do skrzynki pocztowej
- proces wrzuca wiadomość do skrzynki
- proces zakłada, że warstwa transportu dostarczy komunikat do odbiorcy



Adresowanie procesów:

- Żeby proces mógł odbierać komunikaty, musi mieć identyfikator
- Każdy host ma unikatowy 32-bitowy adres IP
- **Pytanie:** czy adres IP hosta na którym działa proces wystarczy dla identyfikacji procesu?
- **Odpowiedź:** Nie, wiele procesów może działać na jednym hoście
- Identyfikator zawiera adres IP oraz **numer portu** związany z procesem na hoście
- Przykładowe numery portów:
 - serwer HTTP: 80
 - serwer poczty: 25

Jakiej usługi transportowej potrzebuje aplikacja?

Straty

- niektóre aplikacje (n.p., audio) tolerują pewną ilość strat
- inne aplikacje (n.p., transfer plików, telnet) wymagają 100% niezawodności

Opóźnienie

- niektóre aplikacje (n.p., telefonia Internetowa, gry interaktywne) wymagają niskich opóźnień

Przepustowość

- niektóre aplikacje (n.p., audio/video) wymagają minimalnej przepustowości
- inne aplikacje (“elastyczne”) mogą wykorzystać tyle przepustowości, ile otrzymają

Wymagania aplikacji dotyczące transportu

Aplikacja	Straty	Przepustowość	Opóźnienie
transfer plików	bez strat	elastyczna	nie
poczta elektroniczna	bez strat	elastyczna	nie
WWW	bez strat	elastyczne	nie
audio/video w czasie rzeczywistym	toleruje straty	audio: 5kbps-1Mbps video: 10kbps-5Mbps	tak, setki ms
odtwarzane audio/video	toleruje straty	jak wyżej	tak, kilka s
gry interaktywne	toleruje straty	powyżej kilku kbps	tak, setki ms
komunikatory	bez strat	elastyczna	tak i nie

Wymagania aplikacji dotyczące transportu

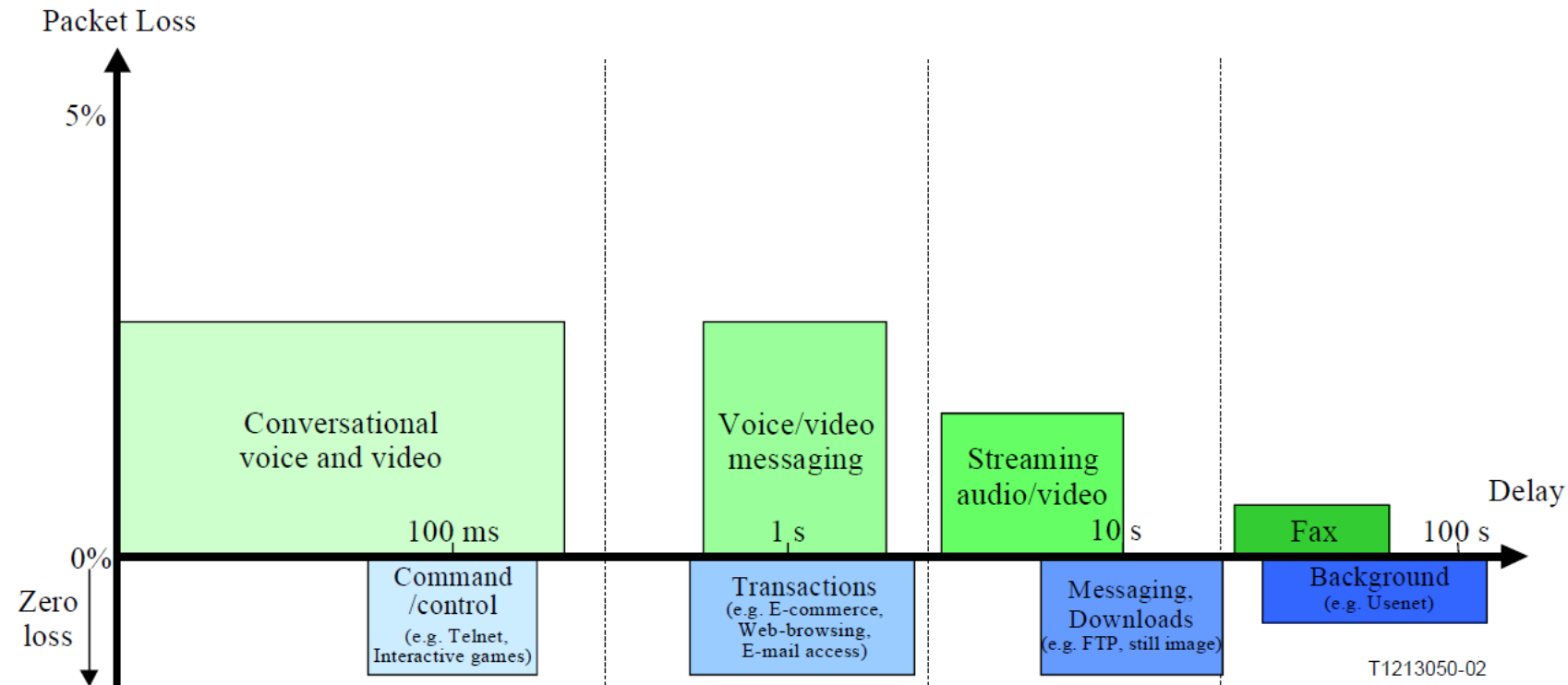


Figure 1/G.1010 – Mapping of user-centric QoS requirements

Usługi transportowe Internetu

usługa TCP:

- *połączeniowa*: wymaga inicjalizacji połączenia pomiędzy procesami
- *niezawodna komunikacja* pomiędzy procesami
- *kontrola przepływu*: nadawca nie przeciąży odbiorcy
- *kontrola przeciążenia*: nadawca zwolni, jeśli sieć zostanie przeciążona
- *nie udostępnia*: informacji o czasie, gwarancji minimalnej przepustowości

usługa UDP:

- zawodna komunikacja pomiędzy procesami
- nie udostępnia: tworzenia połączenia, niezawodności, kontroli przepływu, kontroli przeciążenia, informacji o czasie, ani gwarancji przepustowości

Pytanie: Po co? Czemu istnieje usługa UDP?

Aplikacje Internetowe: protokoły warstw aplikacji i transportu

Aplikacja	Protokół warstwy aplikacji	Wykorzystywany protokół transportowy
e-mail	SMTP [RFC 2821]	TCP
zdalny terminal	Telnet [RFC 854]	TCP
WWW	HTTP [RFC 2616]	TCP
transfer plików	FTP [RFC 959]	TCP
media strumieniowe	prywatne (n.p. RealNetworks)	TCP lub UDP
Telefonia Internetowa	prywatne (n.p., Dialpad)	zwykle UDP

Mapa Wykładu

- Zasady budowy protokołów w. aplikacji
- WWW i HTTP
- DNS
- Poczta elektroniczna
- SMTP, POP3, IMAP
- FTP
- Dystrybucja zawartości
- Schowki Internetowe
- Sieci dystrybucji zawartości

WWW i HTTP

Najpierw terminologia

- Strona WWW składa się z obiektów
- Obiekt może być plikiem HTML, obrazem JPEG, apletem Java, plikiem audio,...
- Strona WWW składa się z bazowego pliku HTML który zawiera szereg odnośników do obiektów
- Każdy obiekt posiada adres URL (Uniform Resource Locator)
- Przykładowy URL:

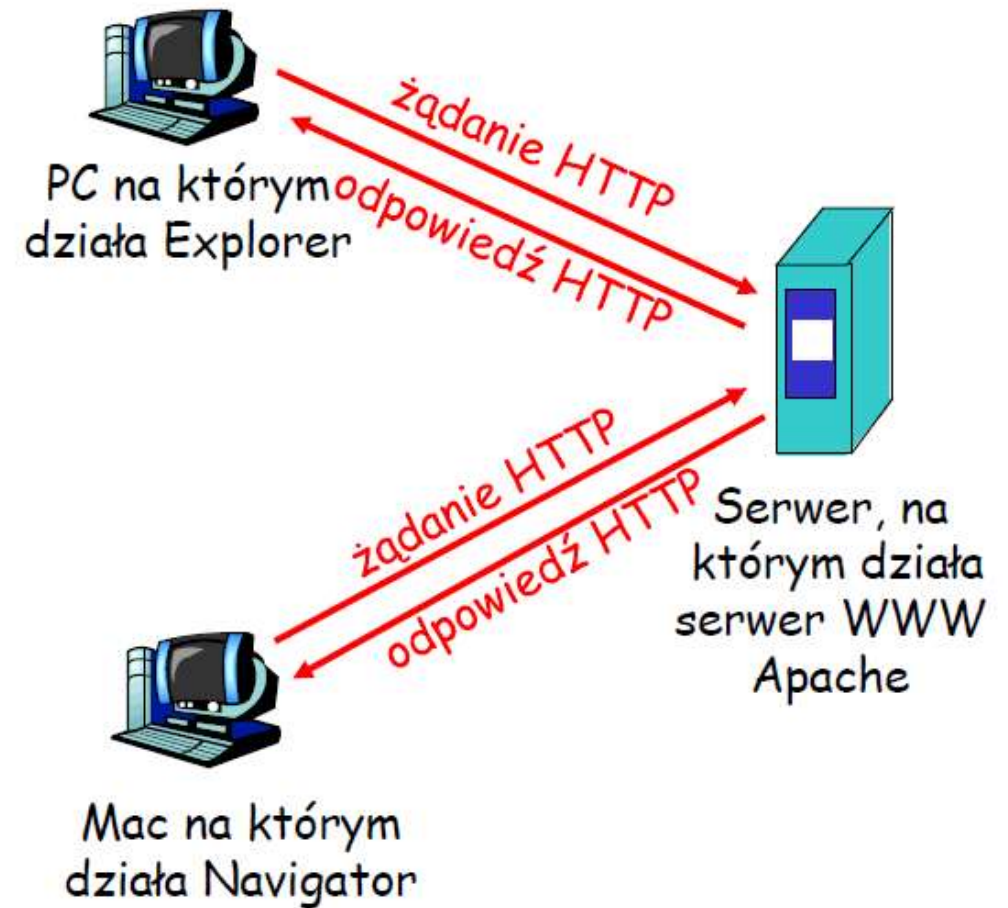
`www.szkoła.edu.pl/katedra/pic.gif`

nazwa hosta ścieżka

Przegląd HTTP

HTTP: hypertext transfer protocol

- Protokół warstwy aplikacji WWW
- model klient/serwer
- *klient*: przeglądarka żąda, otrzymuje, “wyświetla” obiekty WWW
- *serwer*: serwer WWW wysyła obiekty w odpowiedzi na żądania
- HTTP 1.0: RFC 1945
- HTTP 1.1: RFC 2068



Przegląd HTTP (c.d.)

Używa TCP:

- klient nawiązuje połączenie TCP (tworzy gniazdo) do serwera, port 80
- serwer przyjmuje połączenie TCP od klienta
- przeglądarka (klient HTTP) i serwer WWW (serwer HTTP) wymieniają komunikaty HTTP (komunikaty protokołu warstwy aplikacji)
- połączenie TCP jest zamykane

HTTP jest “bezstanowy”

- serwer nie utrzymuje i nie wykorzystuje informacji o uprzednich połączeniach HTTP

Dygresja:

Protokoły, które utrzymują “stan”, są złożone!

- *historia (stan) musi być utrzymywana*
- *jeśli serwer lub klient będzie miał awarię, informacje o stanie mogą być niezgodne, muszą zostać ujednolicone*

Połączenie HTTP

Nietrwałe HTTP

- Najwyżej jeden obiekt jest posyłany przez połączenie TCP, które potem zostaje zamknięte.
- HTTP/1.0 używa nietrwałego HTTP

Trwałe HTTP

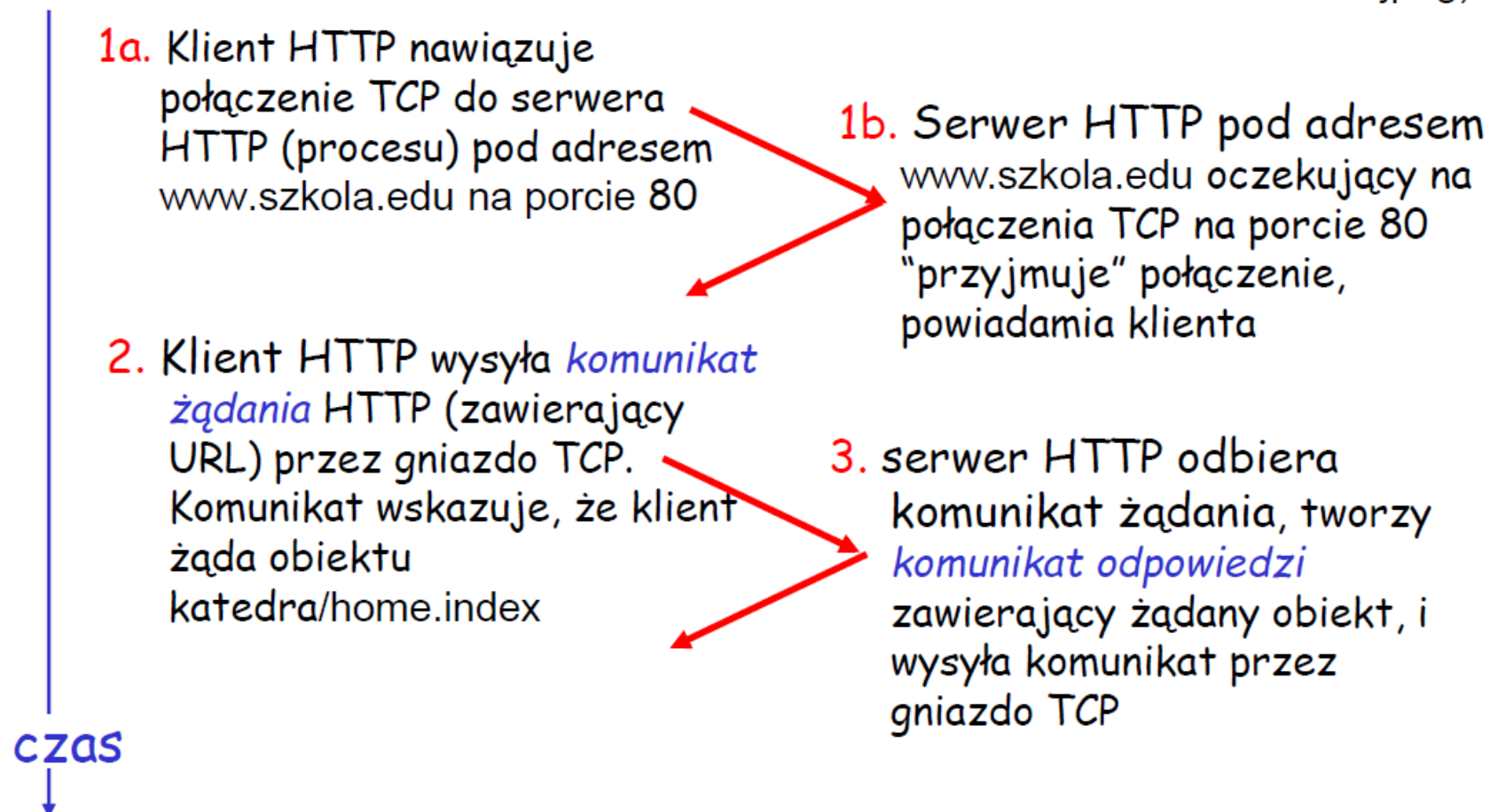
- Wiele obiektów mogą zostać posłane przez jedno połączenie TCP pomiędzy klientem i serwerem.
- HTTP/1.1 domyślnie używa trwałego HTTP

Nietrwałe HTTP

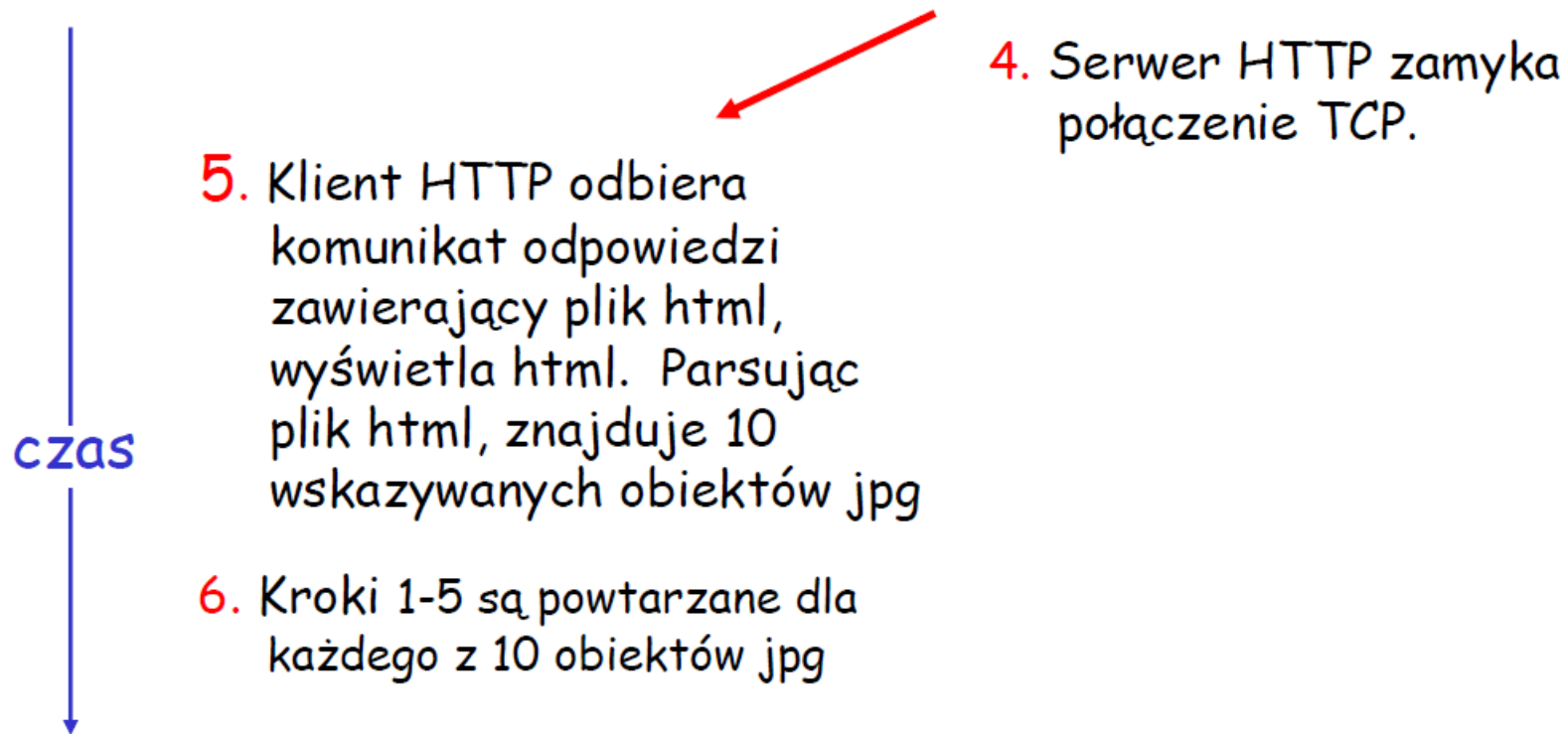
Użytkownik wprowadza adres URL

`www.szkola.edu.pl/katedra/home.index`

(strona zawiera
tekst, wskazania
do 10 obrazów
jpeg)



Nietrwałe HTTP(c.d.)



Modelowanie czasu obsługi: nietrwałe HTTP

Definicja RTT (ang. *Round Trip Time*): czas potrzebny na przesłanie małego pakietu od nadawcy do odbiorcy i z powrotem.

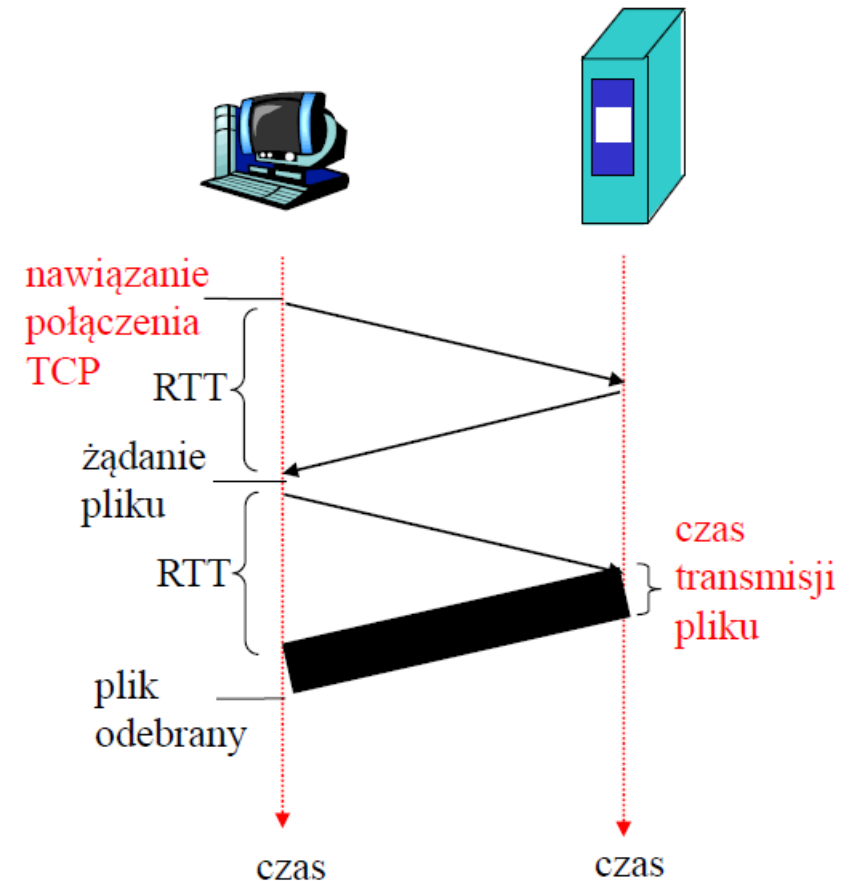
Czas obsługi:

- 1 RTT na część inicjacji połączenia TCP
- 1 RTT na przesłanie żądania HTTP i na powrót pierwszych bajtów odpowiedzi HTTP
- czas transmisji pliku

razem = 2 RTT + czas transmisji

- Czas pobrania strony z 10 obrazkami:

20 RTT + czas transmisji plików



Trwałe HTTP

Nietrwałe HTTP:

- wymaga 2 RTT na każdy obiekt
- System operacyjny hosta musi pracować i udostępniać zasoby na każde połączenie TCP
- powolny start TCP
- jednak przeglądarki często otwierają równoległe połączenia TCP

Trwałe HTTP:

- serwer zostawia otwarte połączenie po wysłaniu odpowiedzi
- następne komunikaty HTTP pomiędzy tym samym klientem i serwerem na tym samym połączeniu TCP

Trwałe HTTP bez grupowych żądań:

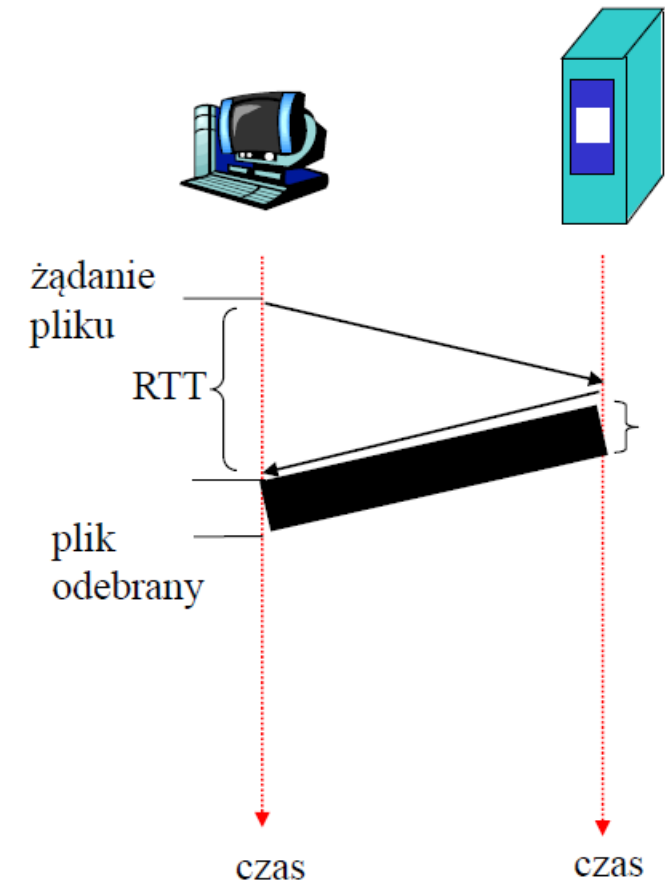
- klient wysyła nowe żądanie dopiero, gdy odebrał poprzednią odpowiedź
- 1 RTT na każdy żądany obiekt

Trwałe z grupowymi żadaniami:

- domyślne w HTTP/1.1
- klient wysyła żądanie jak tylko znajdzie w stronie wskazany obiekt
- tylko jeden RTT dla wszystkich żądanych obiektów

Czas obsługi dla trwałego HTTP

- Dla każdego obiektu na stronie:
 - Jedno RTT na wysłanie żądania HTTP i otrzymanie odpowiedzi
 - Czas transmisji pliku
- Oszczędzamy jedno RTT dla każdego obiektu na nawiązywaniu połączenia TCP
- Dla strony zawierającej 10 obrazków:
10 RTT + czas transmisji plików



Ciasteczka: utrzymywanie “stanu”

Wiele znanych portali WWW używa ciasteczek

- **Cztery składniki:**

- 1) nagłówek cookie w odpowiedzi HTTP
- 2) nagłówek cookie w żądaniu HTTP
- 3) plik z ciasteczkami na hoście klienta, zarządzany przez przeglądarkę klienta
- 4) baza danych na serwerze WWW

Przykład:

- Ania ma dostęp do Internetu z zawsze tego samego komputera
- Odwiedza pewien portal e-commerce po raz pierwszy
- Gdy pierwsze żądanie HTTP przybywa do portalu, portal tworzy unikalny identyfikator i wpis w bazie danych na serwerze

Ciasteczka (c.d.)

- Co można dzięki ciasteczkom:

- uwierzytelnienie
- wózki z zakupami
- rekomendacje
- stan sesji użytkownika (np. w banku elektronicznym)

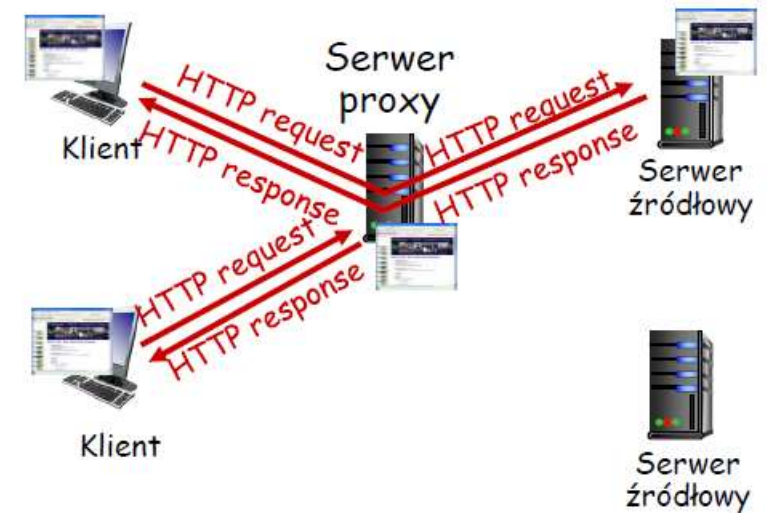
- Ciasteczka a prywatność:

- ☐ ciasteczka pozwalają portalom dowiedzieć się wiele o użytkownikach
- ☐ na niektórych portalach, podaje się nazwisko i adres poczty elektronicznej
- ☐ poprzez ciasteczka, wyszukiwarki mogą poznać więcej informacji
- ☐ firmy marketingowe uzyskują informacje z wielu portali i łączą je

Serwery pośredniczące i schowki WWW

Cel: obsłużyć żądanie klienta bez komunikacji z serwerem źródłowym

- Użytkownik konfiguruje przeglądarkę, żeby używała *serwera pośredniczącego (proxy)*
- Lub żądanie użytkownika jest przekierowywane do serwera pośredniczącego
- Przeglądarka wysyła wszystkie żądania HTTP do serwera pośredniczącego
 - *jeśli* obiekt jest w schowku: wysyłamy go klientowi
 - *Jeśli nie*, proxy żąda obiektu od serwera źródłowego, umieszcza obiekt w schowku, i wysyła go klientowi
- Co jeszcze mogą robić serwery pośredniczące?
 - Skanowanie w poszukiwaniu wirusów
 - Cenzura



HTTP – dalsze ulepszenia

- **HTTP/2:** zwiększa elastyczność serwera w wysyłaniu odpowiedzi do klienta:
- metody, kody statusów, większość nagłóweków nie zmienia się w porównaniu do HTTP 1.1
- Kolejność wysyłanych obiektów zależy od priorytetów określonych przez klienta (niekoniecznie FCFS)
- Możliwość wysyłania obiektów do klienta bez jego żądania (*push*)
- Podział obiektów na “ramki”, wysyłanie ramek w kolejności, która unika blokowania HOL
- **HTTP/3:** dodaje ochronę informacji, funkcje niezawodności i kontroli przeciążenia działają dla każdego obiektu z osobna, korzysta z protokołu UDP

- Zasady budowy protokołów w. aplikacji
- WWW i HTTP
- DNS
- Poczta elektroniczna
- SMTP, POP3, IMAP
- FTP
- Dystrybucja zawartości
- Schowki Internetowe
- Sieci dystrybucji zawartości

DNS: Domain Name System

- **Ludzie:** wiele identyfikatorów:
 - PESEL, nazwisko, numer paszportu
 - **Hosty, routery Internetu:**
 - adres IP (32 bity) – używany do adresowania pakietów
 - **Czy to wystarczy? Co zrobić, jeśli adres IP musi ulec zmianie?**
 - **Jak określać usługi, które są realizowane przez wiele serwerów?**
 - **Jak odróżniać różne usługi, które są realizowane przez jeden serwer?**
- **Domain Name System:**
 - ☐ *rozproszona baza danych* implementowana przez hierarchię wielu *serwerów nazw*
 - ☐ *protokół warstwy aplikacji* hosty, routery, serwery nazw komunikują się, żeby *tłumaczyć* nazwy
 - ☐ uwaga: jedna z głównych funkcji Internetu, implementowana jako protokół w warstwie aplikacji
 - ☐ złożoność na "brzegu" sieci

Rozwiązanie:

“nazwa”, n.p., www.eu.edu.pl – używana przez ludzi

Pytanie: jak tłumaczyć pomiędzy adresami IP i nazwami?

Serwery nazw DNS

Czemu nie scentralizować DNS?

- zagrożenie pojedynczą awarią
- ilość ruchu
- odległość od scentralizowanej bazy
- aktualizacje taki projekt nie jest *skalowalny!*

Zasada delegacji

- organizacja zarządza strefą nazw
- w obrębie strefy, może wydzielać mniejsze strefy
- organizacja przekazuje zarządzanie za strefę innym organizacjom

- żaden serwer nie zna wszystkich odwzorowań adresów IP i nazw DNS

lokalne serwery nazw:

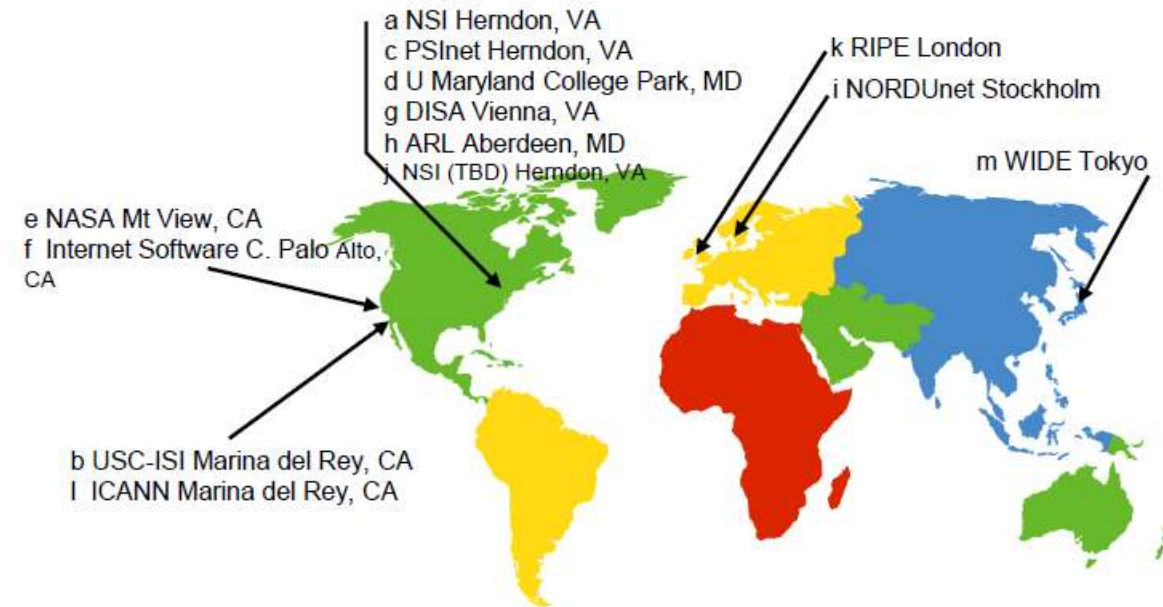
- każdy DI, organizacja ma *lokalny (domyślny) serwer nazw*
- pytanie DNS z hosta jest kierowane najpierw do lokalnego serwera nazw

autorytatywny serwer nazw:

- dla hosta: przechowuje adres IP i nazwę DNS hosta
- może dokonać odwzorowania pomiędzy nazwą i adresem dla tego hosta

DNS: serwery u korzenia

- lokalny serwer nazw pyta serwer u korzenia, gdy nie może przetłumaczyć nazwy
- serwer u korzenia:
 - kontaktuje się z serwerem autorytatywnym, jeśli nie zna odwzorowania nazwy
 - otrzymuje odwzorowanie
 - przekazuje odwzorowanie do lokalnego serwera nazw



13 serwerów u
korzenia na całym
świecie

Serwery TLD oraz autorytatywne

- Serwery Top-level domain (TLD):

- Odpowiedzialne za domeny: com, org, net, edu, etc, oraz domeny krajów uk, fr, ca, jp.
- *Network Solutions* utrzymuje serwery dla domeny com
- *Educause* dla domeny edu

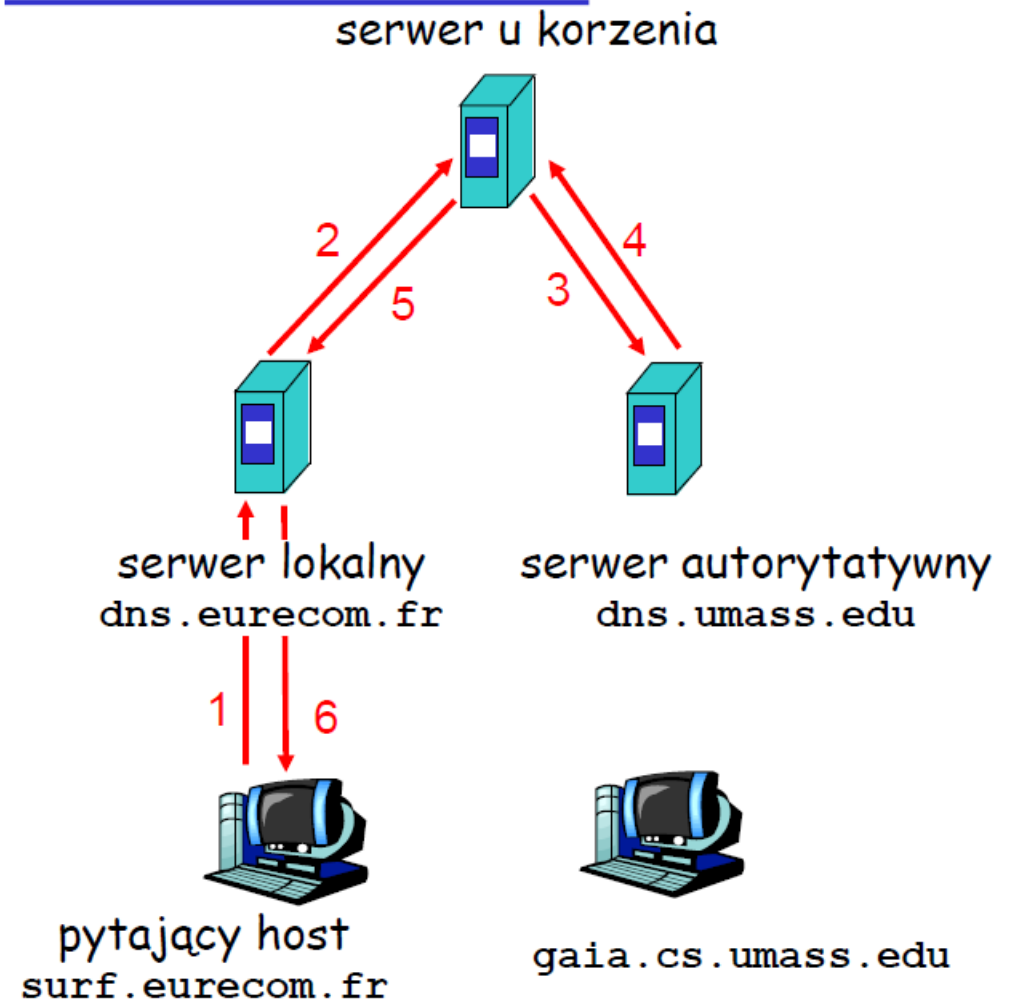
- Autorytatywne serwery DNS:

- Serwery DNS należące do organizacji, udostępniające autorytatywne odwzorowanie adresów DNS na IP dla serwerów tej organizacji (n.p., WWW, poczta).
- Mogą być utrzymywane przez organizację lub jej dostawcę Internetu

Prosty przykład działania DNS

host **surf.eurecom.fr**
potrzebuje adresu IP
gaia.cs.umass.edu

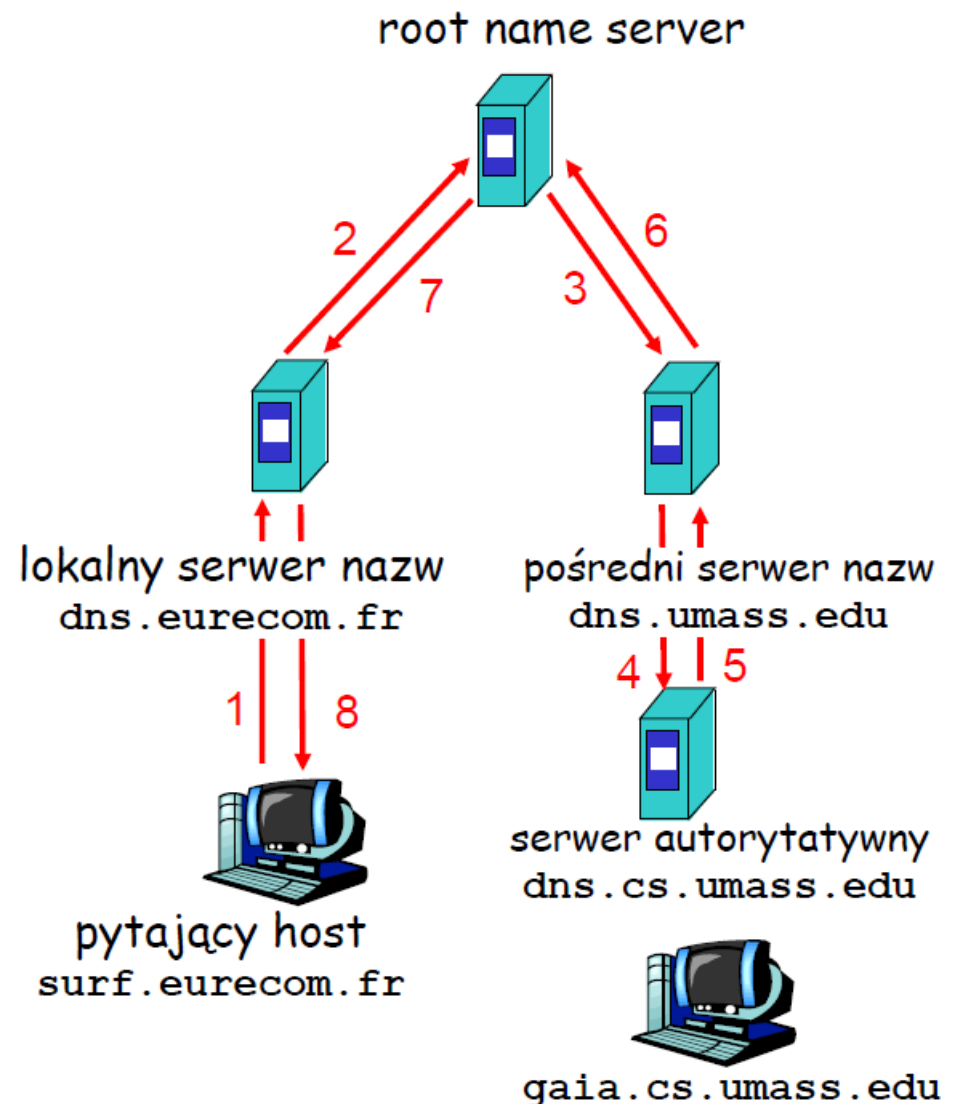
- 1. pyta swój lokalny serwer DNS, **dns.eurecom.fr**
- 2. **dns.eurecom.fr** pyta serwer u korzenia, jeśli to konieczne
- 3. serwer u korzenia pyta serwer autorytatywny, **dns.umass.edu**, jeśli to konieczne



Przykład działania DNS

Serwer u korzenia:

- może nie znać serwera autorytatywnego
- może znać *pośredni serwer nazw*
nazw: kogo spytać o autorytatywny serwer



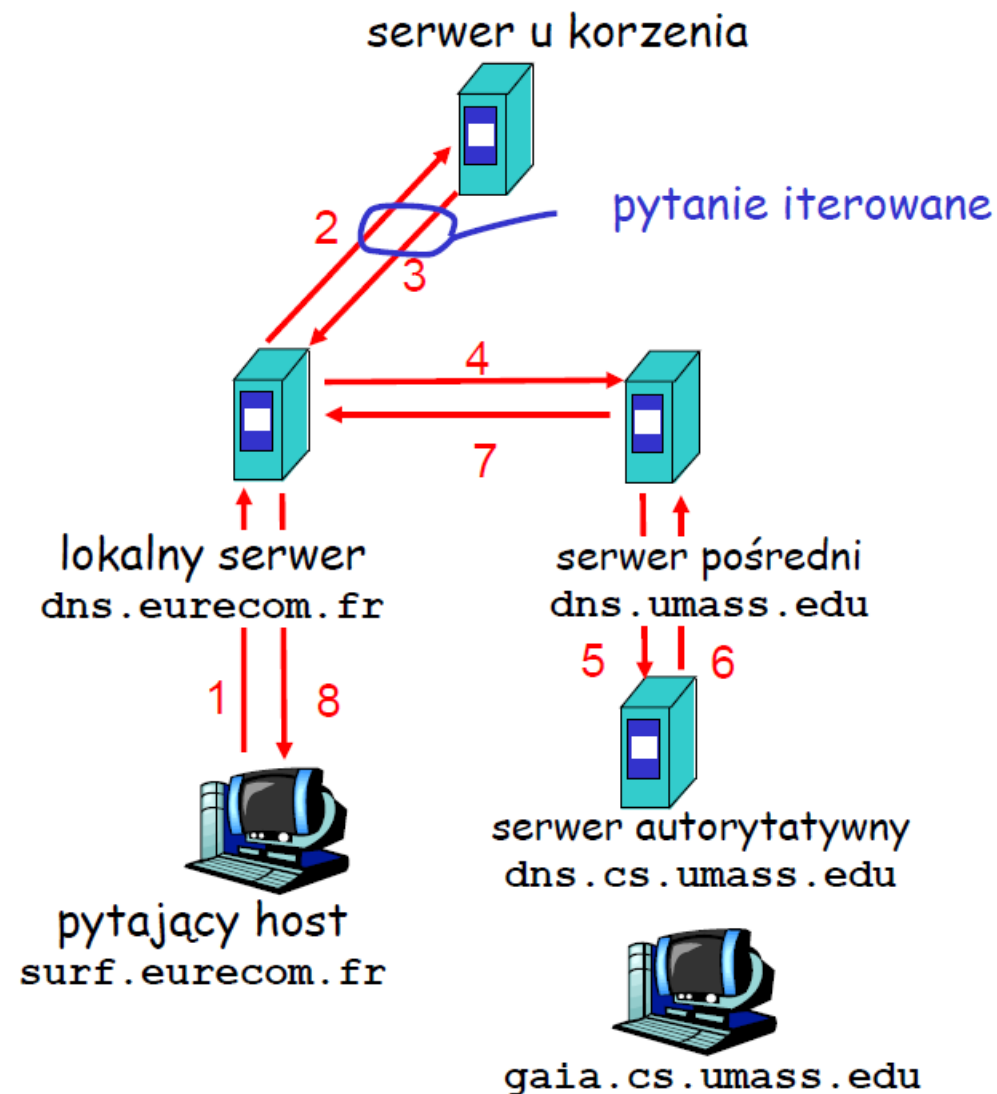
DNS: iterowane pytania

pytanie rekurencyjne:

- obciąża pytany serwer zadaniem zdobycia odpowiedzi
- duże obciążenie?

pytanie iterowane:

- pytany serwer odpowiada adresem serwera, który należy pytać dalej
- “Nie znam tej nazwy, ale spytaj ten serwer



DNS: schowki i aktualizacja rekordów

- gdy (dowolny) serwer nazw pozna odwzorowanie, *zachowuje je w schowku*
 - pozycje w schowku ulegają dezaktualizacji (znikają) po pewnym czasie
- mechanizmy aktualizacji (powiadamiania) są projektowane przez IETF
 - RFC 2136
 - <http://www.ietf.org/html.charters/dnsind-charter.html>

Rekordy DNS

DNS: rozproszona baza danych przechowująca rekordy zasobów **(RZ)**

format RZ: (nazwa, wartość, typ, czas życia)

Typ=A

- **nazwa** hosta
- **wartość** jest adresem IP

Typ=NS

- **nazwa** jest domeną (np. edu.pl)
- **wartość** jest adresem IP autorytatywnego serwera nazw dla tej domeny

Typ=CNAME

- **nazwa** jest aliasem dla pewnej “kanonicznej” (prawdziwej) nazwy
www.ibm.com jest naprawdę servereast.backup2.ibm.
- **wartość** jest nazwą kanoniczną

Typ=MX

- **wartość** jest nazwą serwera poczty związanego z nazwą

Protokół, komunikaty DNS

- Protokół DNS : komunikaty *pytania* i *odpowiedzi*, oba z tym samym *formatem komunikatu*

nagłówek komunikatu

- **identyfikacja:** 16 bitów na numer pytania, odpowiedź używa tego samego numeru
- **flagi:**
 - pytanie lub odpowiedź
 - żądana rekurencja
 - rekurencja dostępna
 - odpowiedź jest autorytatywna

identyfikator	flagi	12 bajtów
ilość pytań	ilość rekordów	
ilość autorytatywnych rekordów	ilość dodatkowych rekordów	
pytania (zmienna ilość)		
odpowiedzi (zmienna ilość)		
autorytatywne odpowiedzi (zmienna ilość rekordów)		
dodatkowa informacja (zmienna ilość rekordów)		

Nazwy domenowe

- Nowe domeny najwyższego poziomu:
- .AERO, .BIZ, .COM, .COOP, .EDU, .GOV, .INFO, .INT, .MIL, .MUSEUM, .NAME, .NET, .ORG, i .PRO
- Lista nie jest zamknięta i będzie poszerzana
- Obecnie, kontroluje je ICANN
- Poprzednio IANA
- Kontrowersja polityczna dotycząca „kontroli” USA nad Internetem
- ICANN jest organizacją międzynarodową
- Czasami krytykowana przez społeczność Internetową (np. przez AlterNIC)
- Kontrowersje biznesowe i prawne
- Nazwy domenowe są w wielu krajach chronione prawnie

- Zasady budowy protokołów w. aplikacji
- WWW i HTTP
- DNS
- Poczta elektroniczna
- SMTP, POP3, IMAP
- FTP
- Dystrybucja zawartości
- Schowki Internetowe
- Sieci dystrybucji zawartości

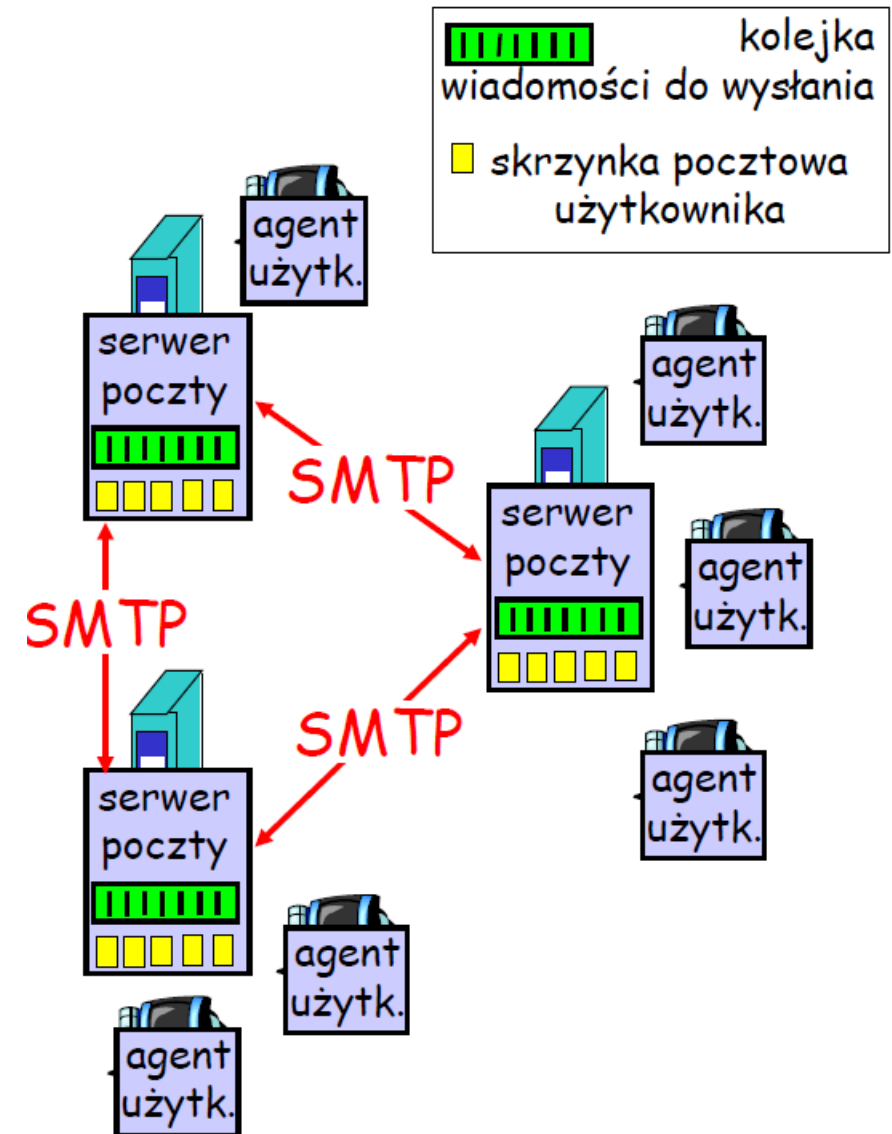
Poczta elektroniczna

Trzy główne składniki:

- agenci użytkownika
- serwery poczty
- simple mail transfer protocol: SMTP

Agent użytkownika (AU)

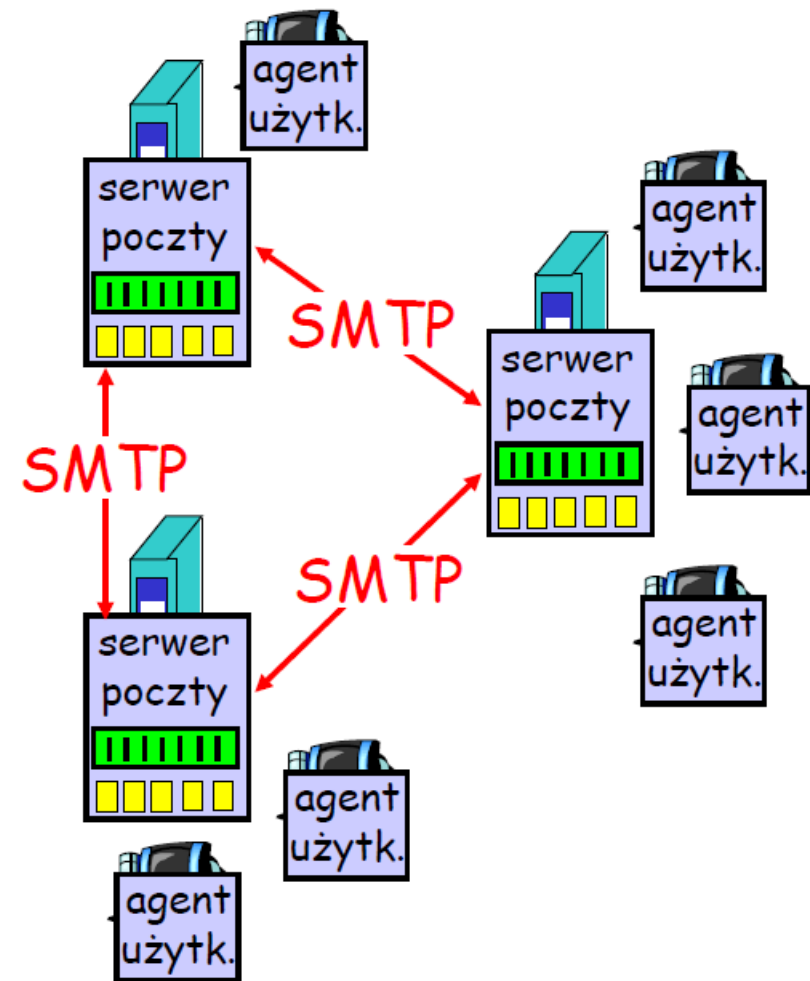
- czyli “przeglądarka poczty”
- kompozycja, edycja, czytanie poczty elektronicznej
- n.p., Eudora, Outlook, Netscape Messenger
- wychodzące i przychodzące wiadomości zachowywane są na serwerze



Poczta elektroniczna: serwery poczty

Serwery poczty

- **skrzynka** zawiera wiadomości przychodzące od użytkowników
- **kolejka wiadomości** zawiera wiadomości do wysłania
- **protokół SMTP** wysyła pocztę pomiędzy serwerami poczty
- tak naprawdę, jest to protokół w modelu partnerskim (ang. *peer-to-peer*)

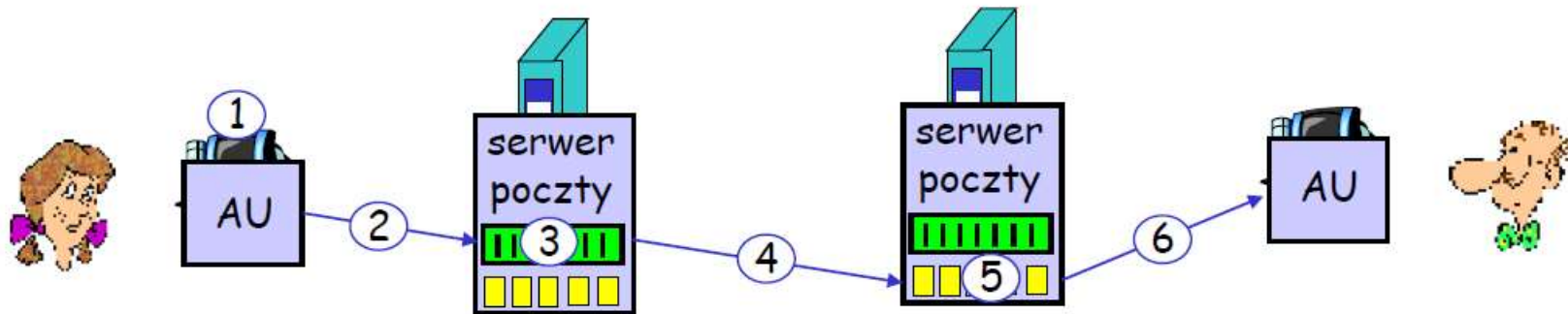


Poczta elektroniczna: SMTP [RFC 2821]

- używa TCP do niezawodnej komunikacji poczty pomiędzy serwerami, port 25
- bezpośrednia komunikacja: serwer nadawcy do serwera odbiorcy
- **trzy etapy komunikacji:**
 - ○ inicjalizacja (powitanie)
 - ○ wymiana komunikatów
 - ○ zakończenie
- **interakcja typu "polecenie/odpowiedź"**
 - ○ polecenia: tekst ASCII
 - ○ odpowiedź: kod i fraza statusu
- komunikaty muszą być kodowane 7-bitowym ASCII

Scenariusz: Alicja wysyła pocztę do Boba

- 1) Alicja używa AU do skomponowania listu i wysyła go do:
`bob@szkola.edu.pl`
- 2) AU Alicji wysyła komunikat do jej serwera poczty; komunikat jest umieszczany w kolejce
- 3) Serwer SMTP otwiera połączenie TCP z serwerem poczty Boba
- 4) Serwer SMTP Alicji wysyła komunikat przez połączenie TCP
- 5) Serwer SMTP Boba umieszcza list w skrzynce Boba
- 6) Bob używa AU do przeczytania wiadomości



Przykładowa interakcja SMTP

- S: 220 hamburger.edu
- C: HELO nalesnik.pl
- S: 250 Hello nalesnik.pl, pleased to meet you
- C: MAIL FROM: <alice@nalesnik.pl>
- S: 250 alice@nalesnik.pl... Sender ok
- C: RCPT TO: <bob@hamburger.edu>
- S: 250 bob@hamburger.edu ... Recipient ok
- C: DATA
- S: 354 Enter mail, end with "." on a line by itself
- C: Czy lubisz ketchup?
- C: A moze ogoreczka?
- C: .
- S: 250 Message accepted for delivery
- C: QUIT
- S: 221 hamburger.edu closing connection

Podsumowanie o SMTP

- SMTP używa trwałych połączeń
- SMTP wymaga, żeby komunikat (nagłówek i dane) były kodowane w 7-bitowym ASCII
- Serwer SMTP używa `CRLF.CRLF` do rozpoznania końca danych

Porównania z HTTP:

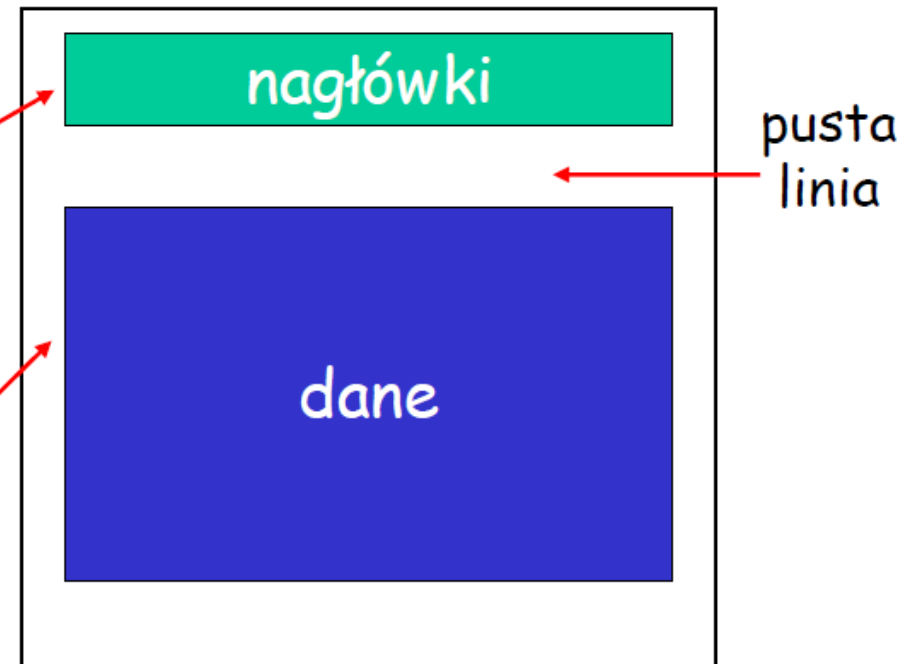
- HTTP: pull (pobieranie)
- SMTP: push (wypychanie)
- Oba mają komunikaty żądań/odpowiedzi w ASCII,
- HTTP: każdy obiekt zawarty w swoim własnym komunikacie odpowiedzi
- SMTP: wiele obiektów może być wysłane w wieloczęściowym komunikacie

Format komunikatu poczty

SMTP: protokół dla poczty elektronicznej

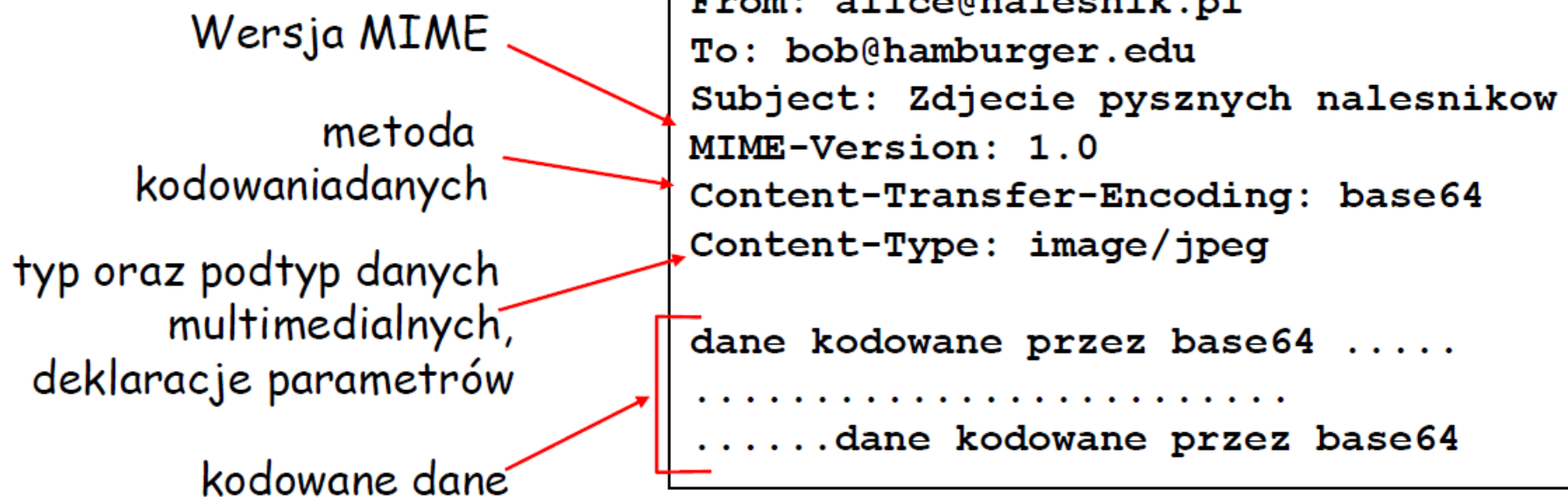
RFC 822: standard opisujący format komunikatów tekstowych:

- ❑ linie nagłówek, n.p.,
 - To:
 - From:
 - Subject:*różne od poleceń SMTP!*
- ❑ dane
 - "list", tylko znaki ASCII



Format komunikatu poczty: rozszerzenia dla multimedialnych

- ❑ MIME: multimedia mail extension, RFC 2045, 2056
- ❑ dodatkowe linie nagłówka określają typ MIME dla zawartości listu



Typy MIME

Content-Type: typ/podtyp; parametry

Tekst

- przykładowe podtypy: `plain`, `html`

Obraz

- przykładowe podtypy: `jpeg`, `gif`

Dźwięk

- przykładowe podtypy: `basic` (kodowanie 8-bit, mu-law), `32kadpcm` (kodowanie 32 kbps)

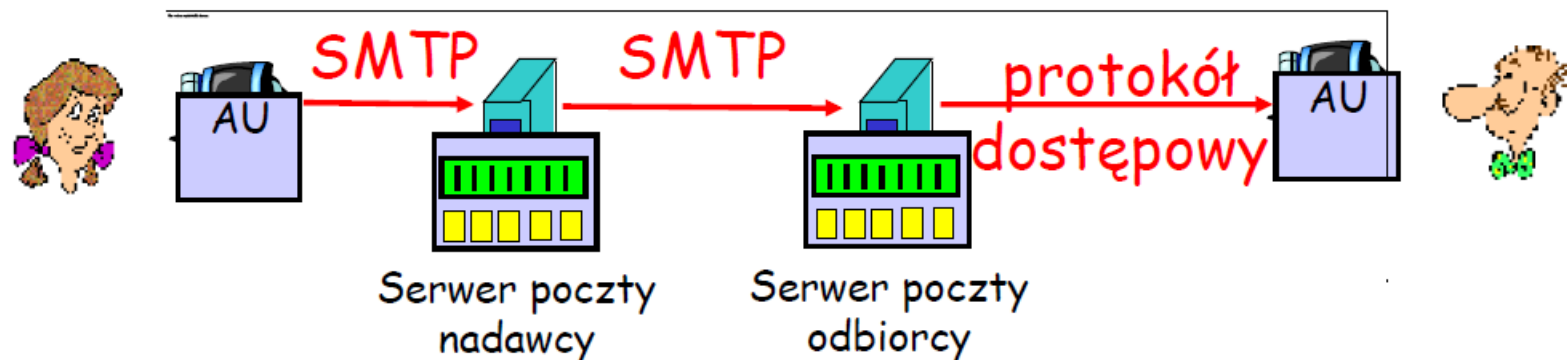
Wideo

- przykładowe podtypy: `mpeg`, `quicktime`

Dane aplikacji

- dane, które muszą zostać przetworzone przez aplikacje, zanim można je pokazać
- przykładowe podtypy: `msword`, `octet-stream`

Protokoły dostępu do poczty



- ❑ SMTP: dostarczanie poczty do serwera odbiorcy
- ❑ Protokół dostępowy: odbieranie poczty z serwera i zarządzanie skrzynką pocztową
 - POP: Post Office Protocol [RFC 1939]
 - uwierzytelnienie (agent <--> serwer) o pobranie poczty
 - IMAP: Internet Mail Access Protocol [RFC 1730]
 - więcej funkcji (bardziej złożony)
 - synchronizacja lokalnej skrzynki oraz skrzynki na serwerze
 - HTTP: Hotmail , Yahoo! Mail, itd.

Protokół POP3

etap uwierzytelnienia

- ❑ polecenia klienta:
 - user: podaję login
 - pass: podaję hasło
- ❑ odpowiedzi serwera
 - +OK
 - -ERR

etap transakcji, klient:

- ❑ list: podaj numery listów
- ❑ retr: pobierz list o numerze
- ❑ dele: usuń
- ❑ quit: zakończ

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass glodny
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

Protokół POP3 (cd) oraz IMAP

- Więcej o POP3

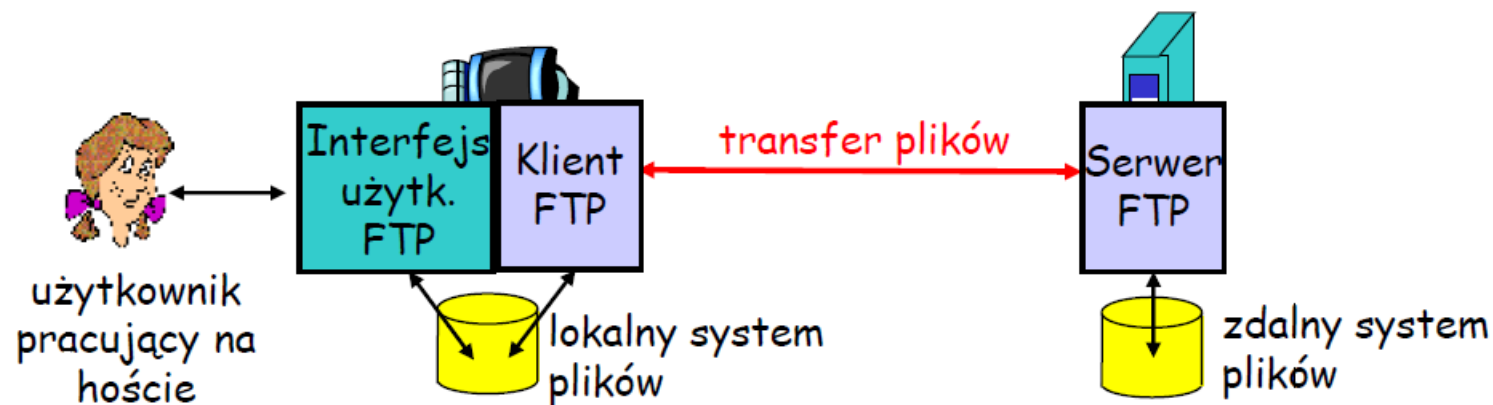
- Poprzedni przykład używał trybu “pobierz i usuń”.
- Bob nie może przeczytać listu ponownie, jeśli zmieni przeglądarkę
- “Pobierz i zostaw”: kopie listów w wielu przeglądarkach
- POP3 jest bezstanowy pomiędzy sesjami

IMAP

- Wszystkie listy są w jednym miejscu: na serwerze
- Użytkownik może organizować pocztę w foldery
- IMAP zachowuje stan użytkownika pomiędzy sesjami:
 - nazwy folderów oraz przyporządkowanie listów do folderów

- Zasady budowy protokołów w. aplikacji
- WWW i HTTP
- DNS
- Poczta elektroniczna
- SMTP, POP3, IMAP
- FTP
- Dystrybucja zawartości
- Schowki Internetowe
- Sieci dystrybucji zawartości

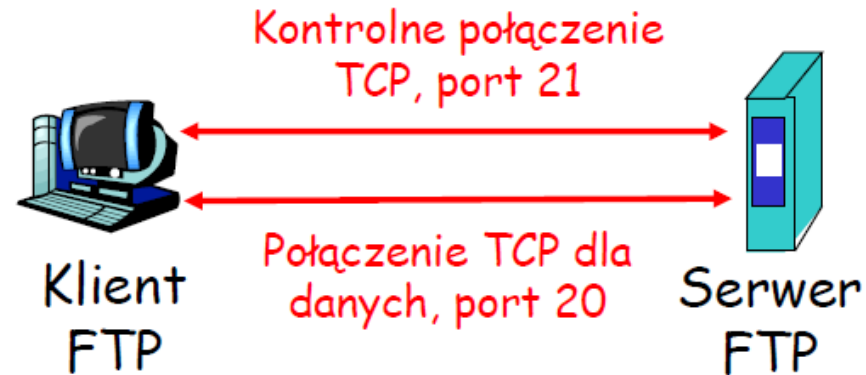
FTP: file transfer protocol



- ❑ transfer pliku z/na zdalny host
- ❑ model klient/serwer
 - *klient*: strona, która rozpoczyna transmisję
 - *serwer*: zdalny host
- ❑ ftp: RFC 959
- ❑ serwer ftp: port 21

FTP: oddzielne połączenie kontrolne i transferu plików

- ❑ Klient FTP kontaktuje się z serwerem na porcie 21 (TCP)
- ❑ Przez połączenie kontrolne, klient uzyskuje autoryzację
- ❑ Klient przegląda zdalny system plików przesyłając polecenia FTP przez połączenie kontrolne.
- ❑ Gdy serwer otrzymuje polecenie transferu pliku, otwiera połączenie TCP do klienta
- ❑ Po przestaniu pliku, serwer zamyka nowe połączenie.



- ❑ Dla przestania drugiego pliku, serwer otwiera drugie połączenie.
- ❑ Połączenie kontrolne: "poza pasmem"
- ❑ Serwer FTP utrzymuje "stan": aktualny katalog, wcześniejszą autoryzację

Polecenia i odpowiedzi FTP

Przykładowe polecenia:

- ❑ posyłane jako tekst ASCII przez połączenie kontrolne
- ❑ USER *login*
- ❑ PASS *password*
- ❑ LIST zwraca listę plików w aktualnym katalogu
- ❑ RETR *nazwaPliku* pobiera plik
- ❑ STOR *nazwaPliku* zapisuje plik na zdalnym hoście

Przykładowe odpowiedzi FTP

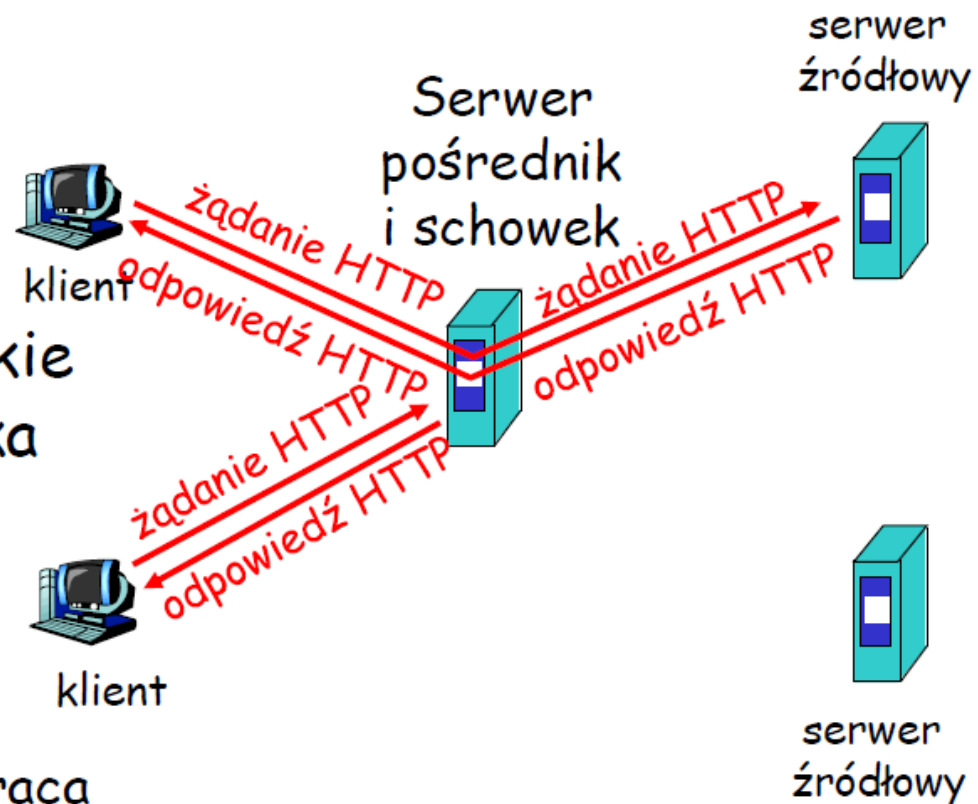
- ❑ kod i opis wyniku (jak w HTTP)
- ❑ 331 Username OK, password required
- ❑ 125 data connection already open; transfer starting
- ❑ 425 Can't open data connection
- ❑ 452 Error writing file

- Zasady budowy protokołów w. aplikacji
- WWW i HTTP
- DNS
- Poczta elektroniczna
- SMTP, POP3, IMAP
- FTP
- Dystrybucja zawartości
- Schowki Internetowe
- Sieci dystrybucji zawartości

Schowki Internetowe (serwery pośredniczące) (ang. Web caches, proxy servers)

Cel: Obsłużyć żądanie klient bez komunikacji z serwerem źródłowym

- użytkownik konfiguruje użycie serwera pośredniczącego w przeglądarce
- przeglądarka śle wszystkie żądania HTTP do schowka
 - jeśli obiekt w schowku: schowek zwraca obiekt
 - jeśli nie, schowek żąda obiektu z serwera źródłowego, następnie zwraca obiekt do klienta



Więcej o schowkach Internetowych

- ❑ Serwer-pośrednik jest zarówno klientem, jak i serwerem
- ❑ Serwer może sprawdzić zgodność obiektu przy pomocy nagłówka HTTP `If-modified-since`
 - Czy schowki powinny ryzykować i zwracać obiekty bez sprawdzania?
 - Używa się heurystyk.
- ❑ Typowo schowki są instalowane przez DI (uniwersytet, firmę, osiedlowego DI)

Po co używa się schowków?

- ❑ Zmniejszają czas oczekiwania na obsługę żądania.
- ❑ Zmniejszają ruch na łączach dostępowych instytucji.
- ❑ Duża ilość schowków w Internecie pozwala "ubogim" dostawcom zawartości na wydajne działanie

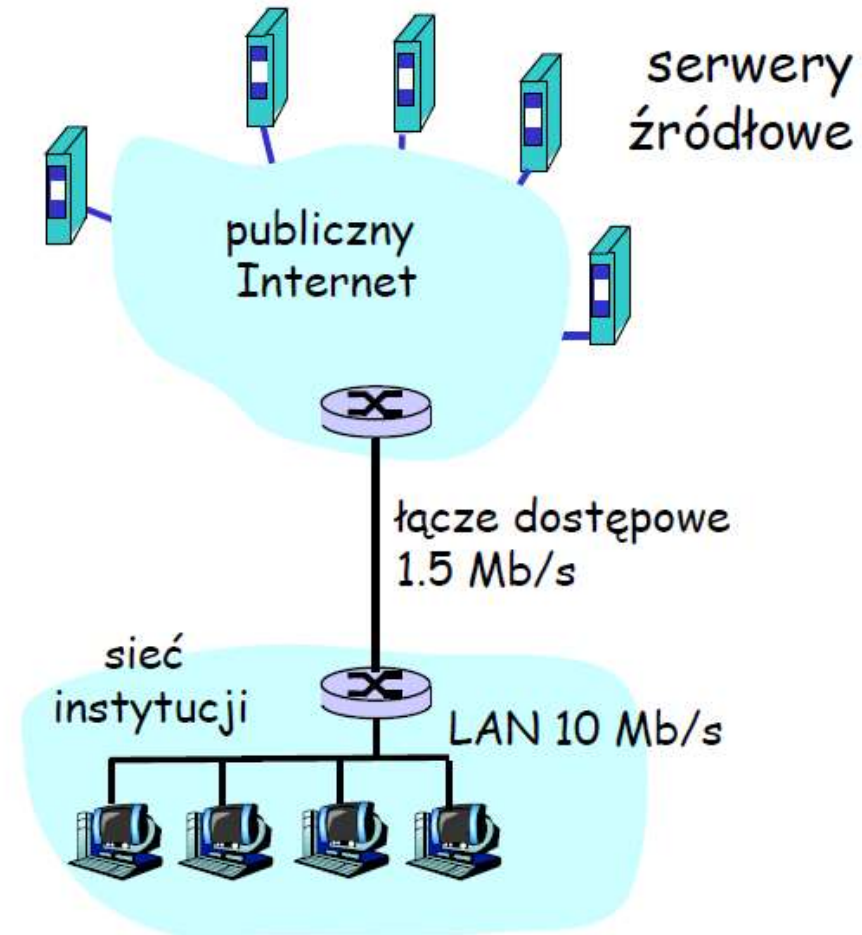
Przykład działania schowka (1)

Założenia

- ❑ średni rozmiar obiektu = 100,000 bitów
- ❑ średnia ilość żądań z sieci instytucji do serwerów źródłowych = 15/sec
- ❑ opóźnienie z sieci instytucji do dowolnego serwera źródłowego i z powrotem = 2 sec

Konsekwencje

- ❑ wykorzystanie sieci LAN = 15%
- ❑ wykorzystanie łącza dostępowego = 100%
- ❑ całkowite opóźnienie = opóźnienie w Internecie + opóźnienie na łączu dostępowym + opóźnienie LAN
= 2 s. + minuty + milisekundy



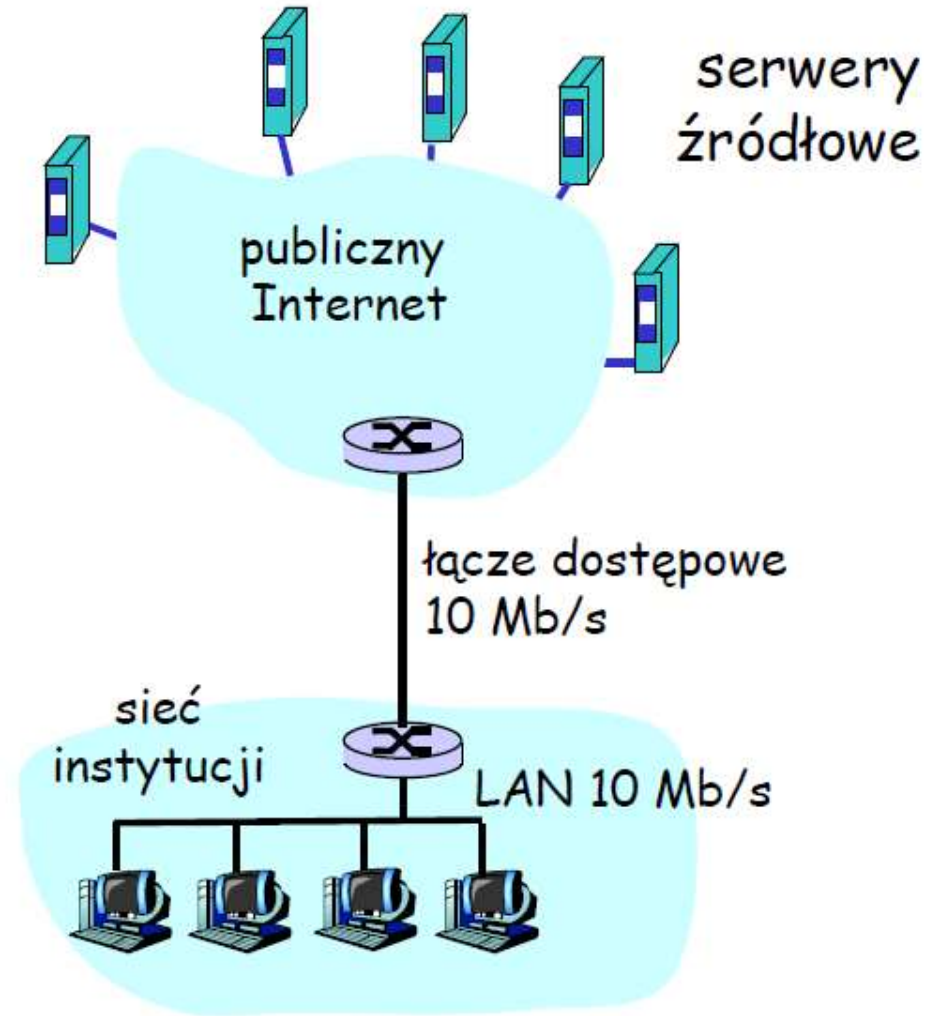
Przykład działania schowka (2)

Możliwe rozwiązanie

- zwiększyć przepustowość łącza dostępowego, do, n.p., 10 Mb/s

Konsekwencje

- wykorzystanie sieci LAN = 15%
- wykorzystanie łącza dostępowego = 15%
- Całkowite opóźnienie = opóźnienie w Internecie + opóźnienie na łączu dostępowym + opóźnienie LAN
 $= 2\text{ s} + \text{ms} + \text{ms}$
- zwiększenie przepustowości jest często bardzo drogie



Przykład działania schowka (3)

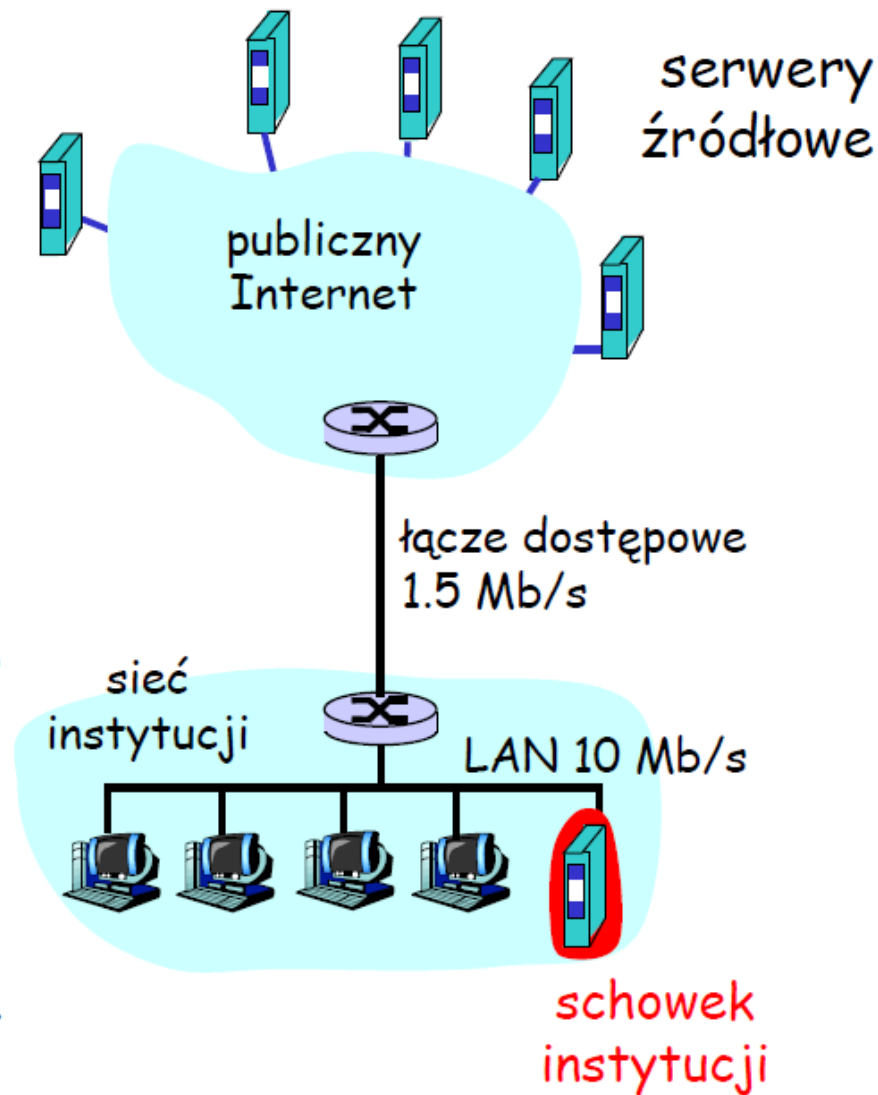
Instalacja schowka

- ❑ założmy częstość trafień 40%

Konsekwencje

- ❑ 40% żądań będzie obsłużona prawie natychmiast
- ❑ 60% żądań nadal obsługiwanych przez serwery źródłowe
- ❑ wykorzystanie łącza dostępowego spada do 60%, co zmniejsza opóźnienia do ok. 10 ms
- ❑ Całkowite opóźnienie =
opóźnienie w Internecie +
opóźnienie na łączu dostępowym +
opóźnienie LAN

$$= 0.6 * 2 \text{ s} + 0.6 * .01 \text{ s} + \text{ms} < 1.3 \text{ s}$$

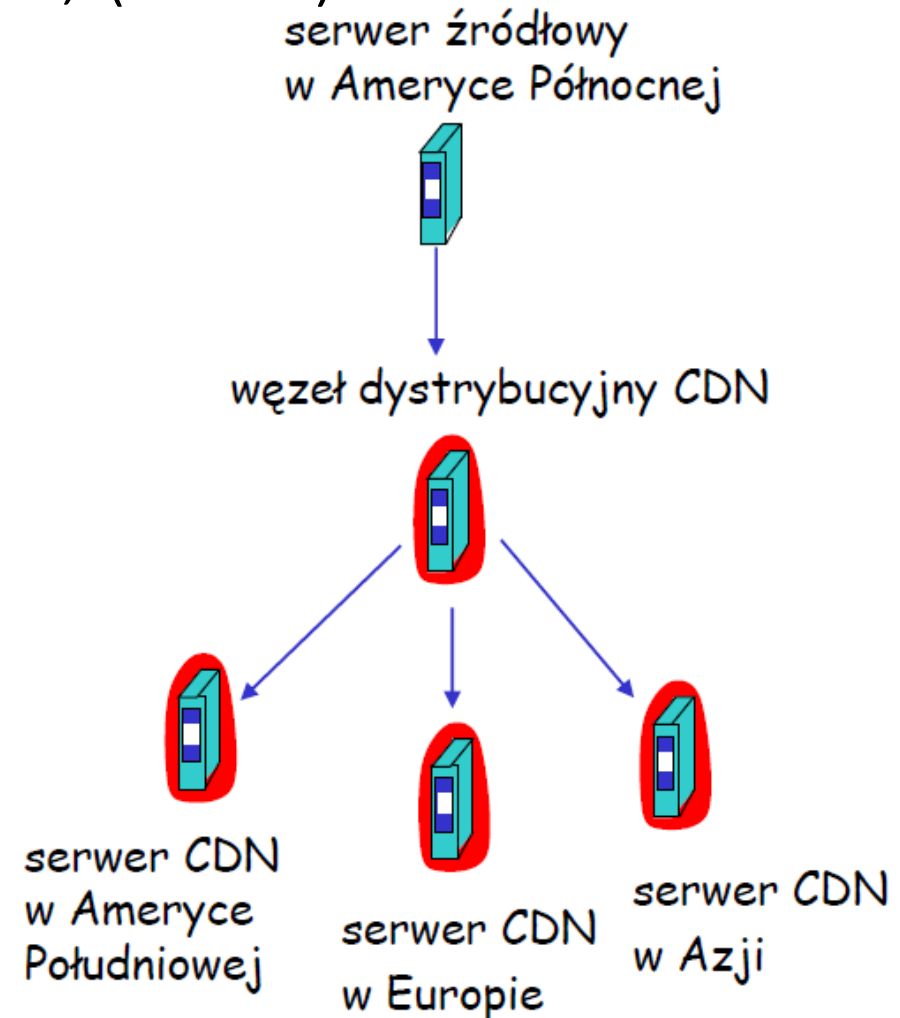


Sieci dystrybucji zawartości ang. Content Distribution Networks, (CDNs)

- ❑ Dostawcy zawartości są klientami sieci CDN.

Replikacja zawartości

- ❑ Firma CDN instaluje setki serwerów CDN w Internecie
 - w DI niższego poziomu, blisko użytkowników
- ❑ CDN replikuje zawartość klientów w swoich serwerach. Gdy dostawca aktualizuje zawartość, CDN aktualizuje serwery



- Koniec